

Dominando el modelo de memoria en arquitecturas CUDA

De la latencia extrema al rendimiento máximo



Nivel Avanzado



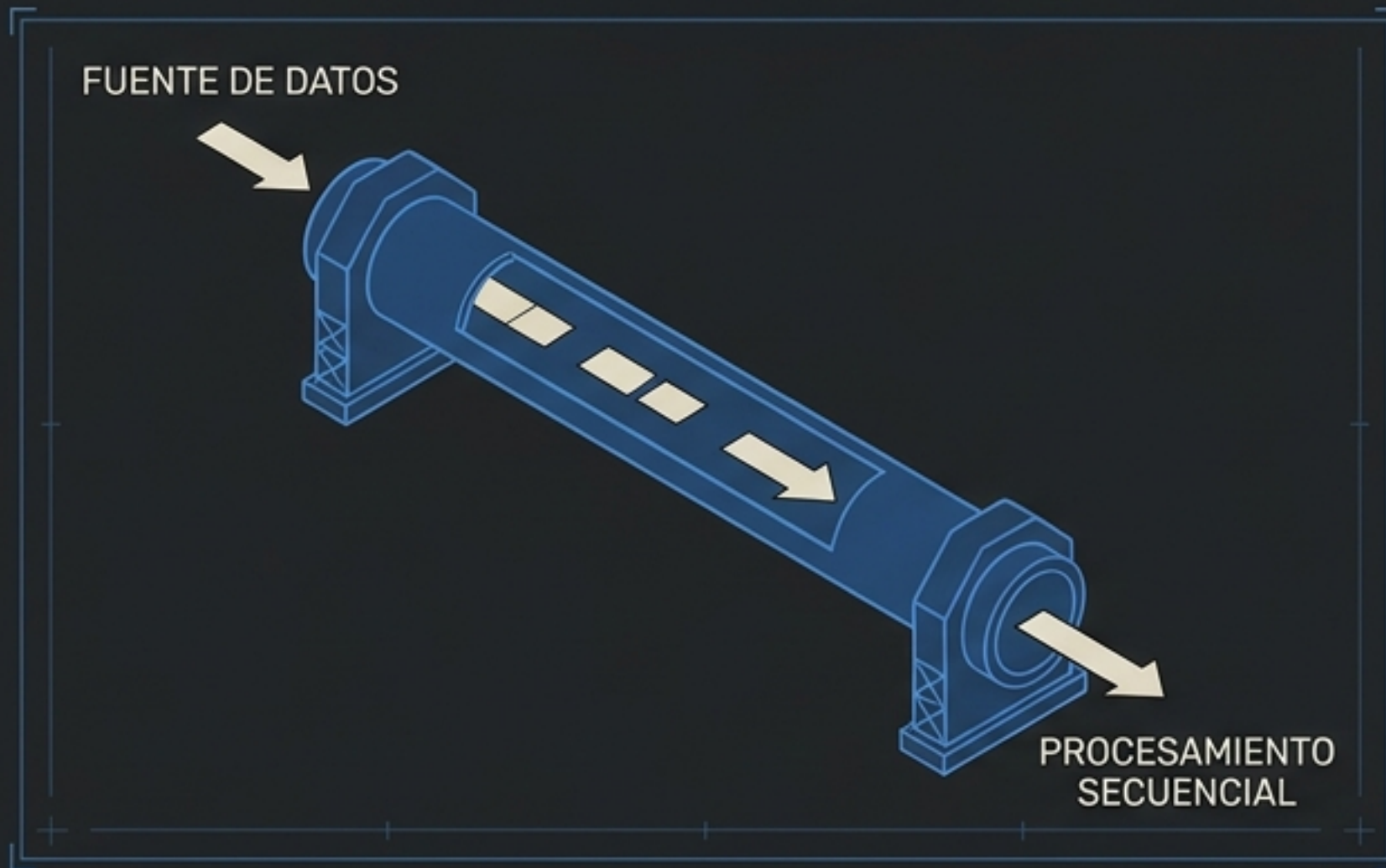
Duración: 2 Horas



Sesión 4

EL USO INCORRECTO DE LA MEMORIA ANULA LA VENTAJA DEL PARALELISMO MASIVO

ENTORNO HOST (CPU)



Flujo de datos secuencial rápido y directo.

ENTORNO DEVICE (GPU)

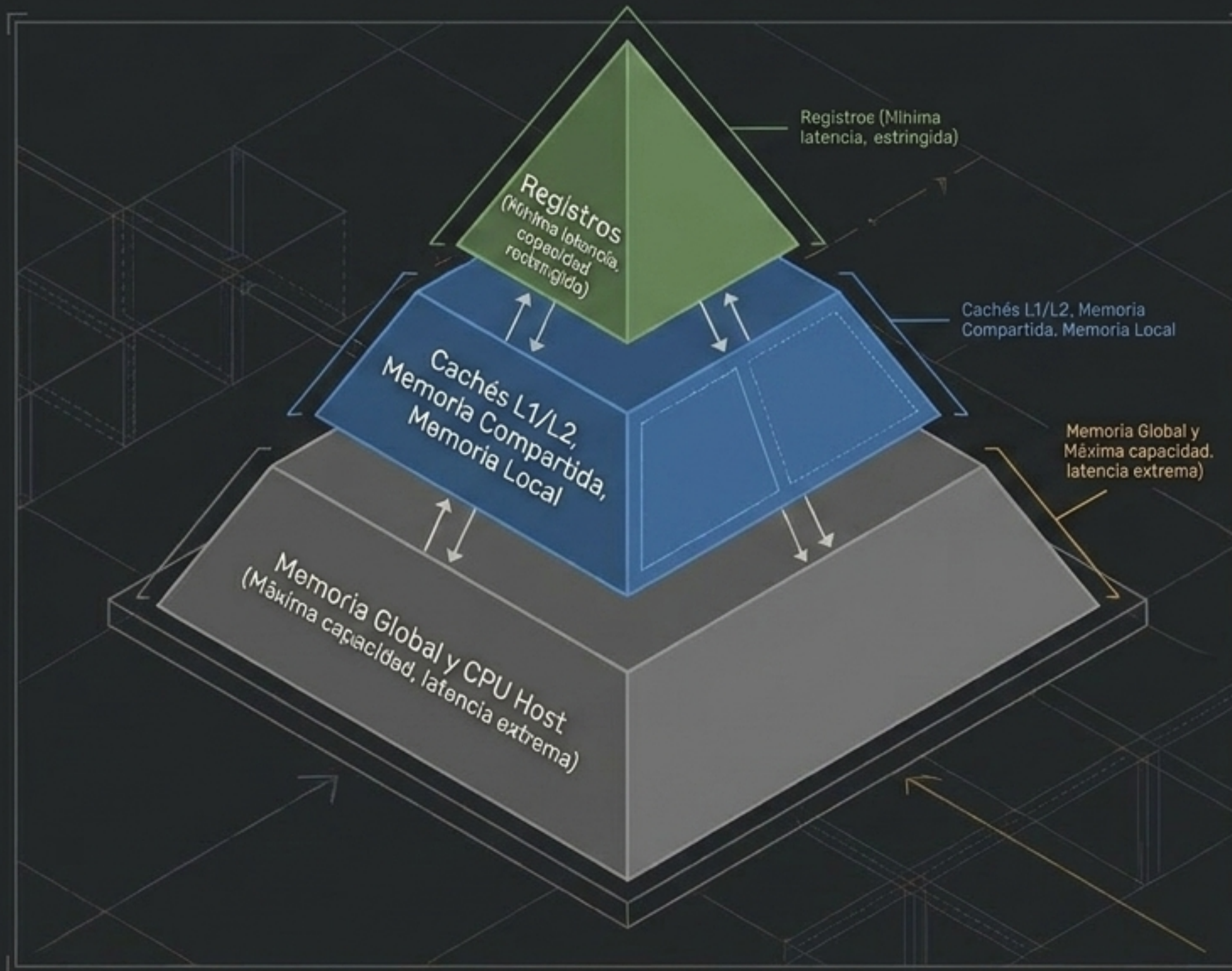


Miles de núcleos masivos esperando ociosos.

LA PARADOJA DE LA INANICIÓN:

Toda la capacidad computacional de los **Streaming Multiprocessors** es inútil si la **velocidad de lectura/escritura no puede alimentarlos**. El se ~~trata de~~ ^{trata de} GPU que ~~esraó un~~ ^{esraó un} ~~hactara en~~ ^{hactara en} ~~quaques~~ ^{quaques} ~~secuciones~~ ^{secuciones} en GPU. El resultado final es un código en GPU que se ejecuta **más lento** que un bucle secuencial en **C++**.

La pirámide de hardware dicta las reglas de supervivencia y optimización



Leyes de Localidad para Mitigación de Latencia

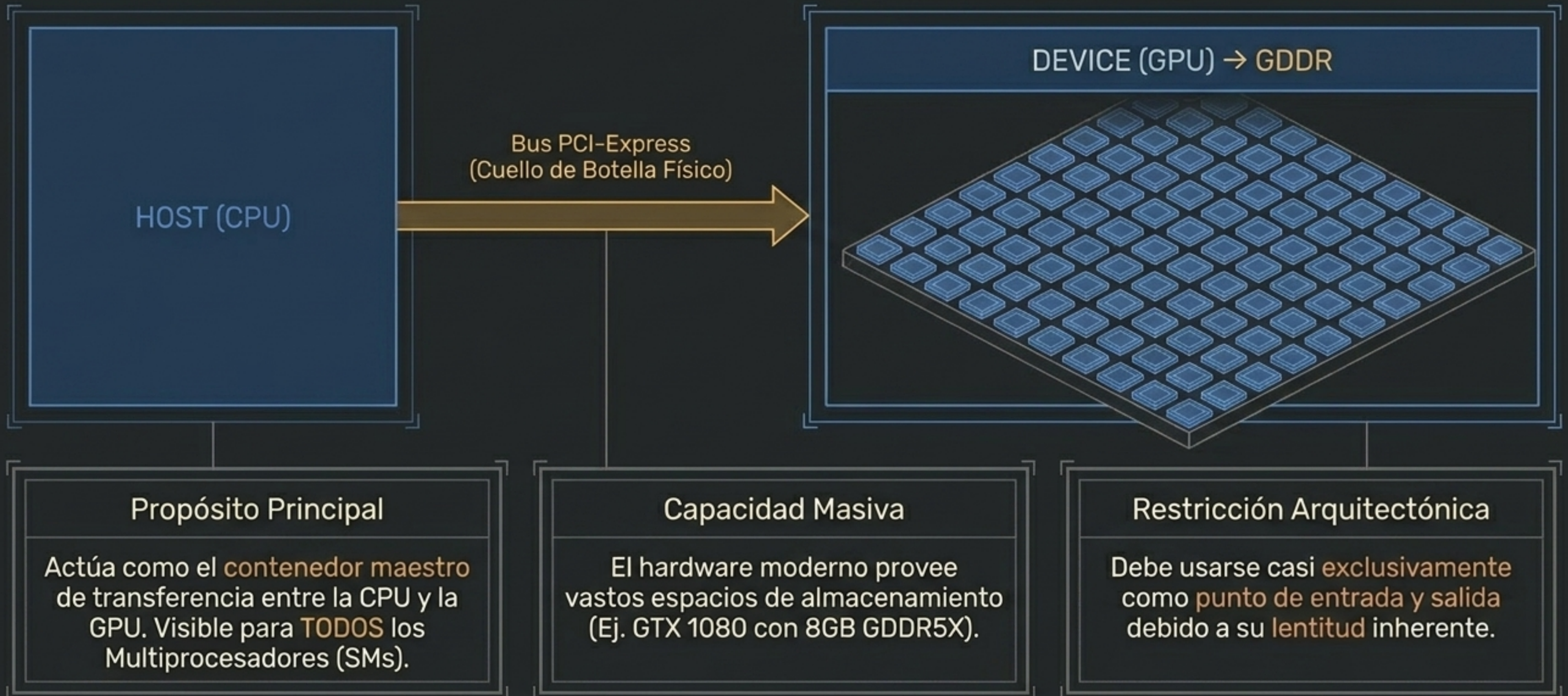
1. Localidad Temporal

Reutilización a corto plazo de un mismo dato procesado.

2. Localidad Espacial

Uso altamente probable de datos adyacentes empaquetados en líneas del bus.

La Memoria Global conecta a todos los hilos, pero penaliza con la máxima latencia



Controlando el contenedor principal mediante la API de memoria en C/C++

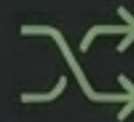
```
cudaMalloc((void**)&dev_a, sizeof(int) * size);  
cudaMemcpy(dev_a, host_a, size, cudaMemcpyHostToDevice);  
cudaMemcpyAsync(...);  
cudaFree(dev_a);
```



Proceso altamente costoso.
Fuerza la sincronización. Ejecutar exclusivamente al inicio o fin del programa.



Síncrono por defecto. Congela la ejecución y el flujo de la CPU.



Variante asíncrona óptima.
Permite solapar transferencias de memoria con la ejecución utilizando Streams.



Limpieza manual estricta y obligatoria para evitar memory leaks.

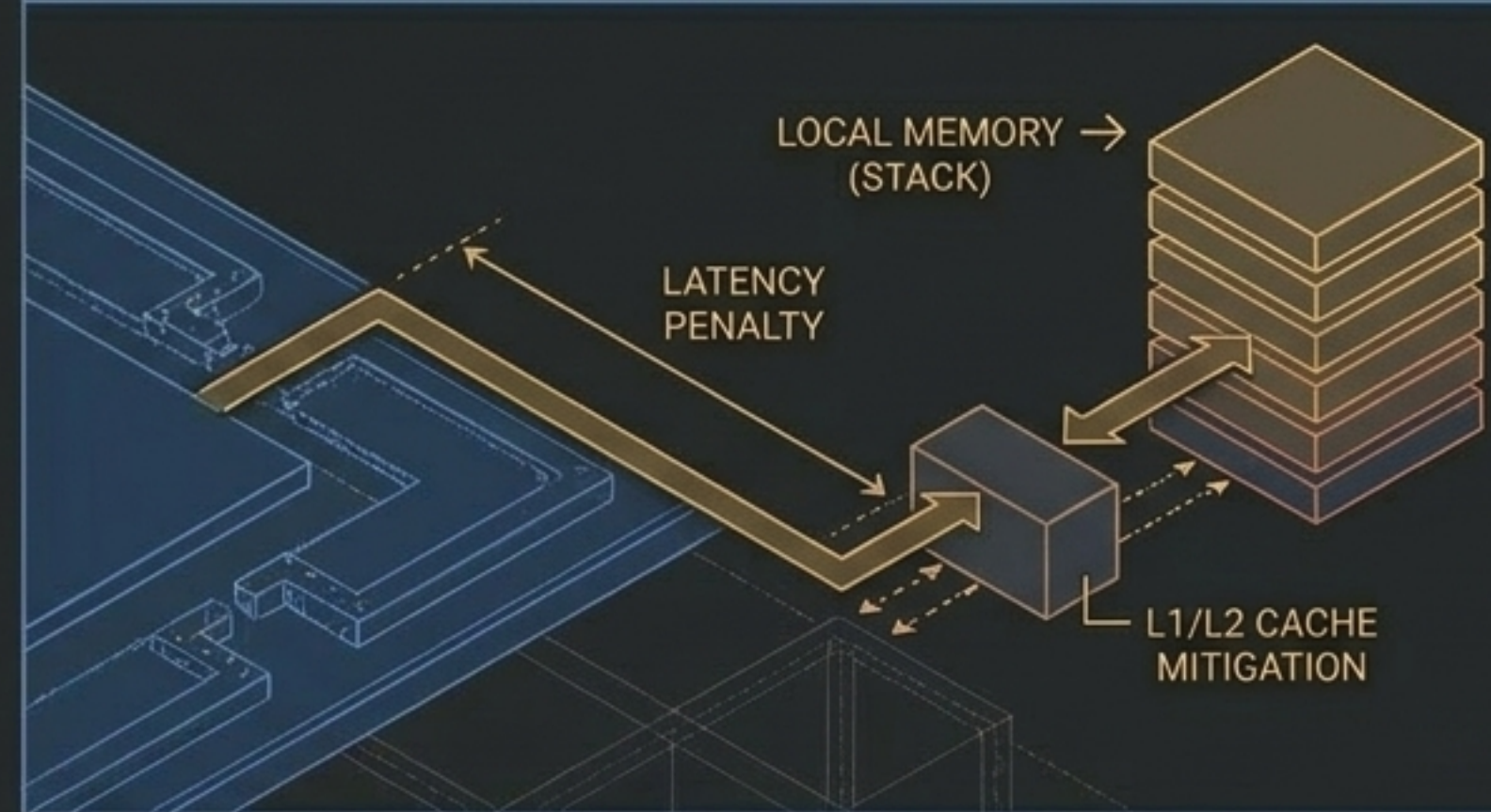
Los registros ofrecen velocidad absoluta si el hilo respeta sus límites físicos

Registros (Registers)



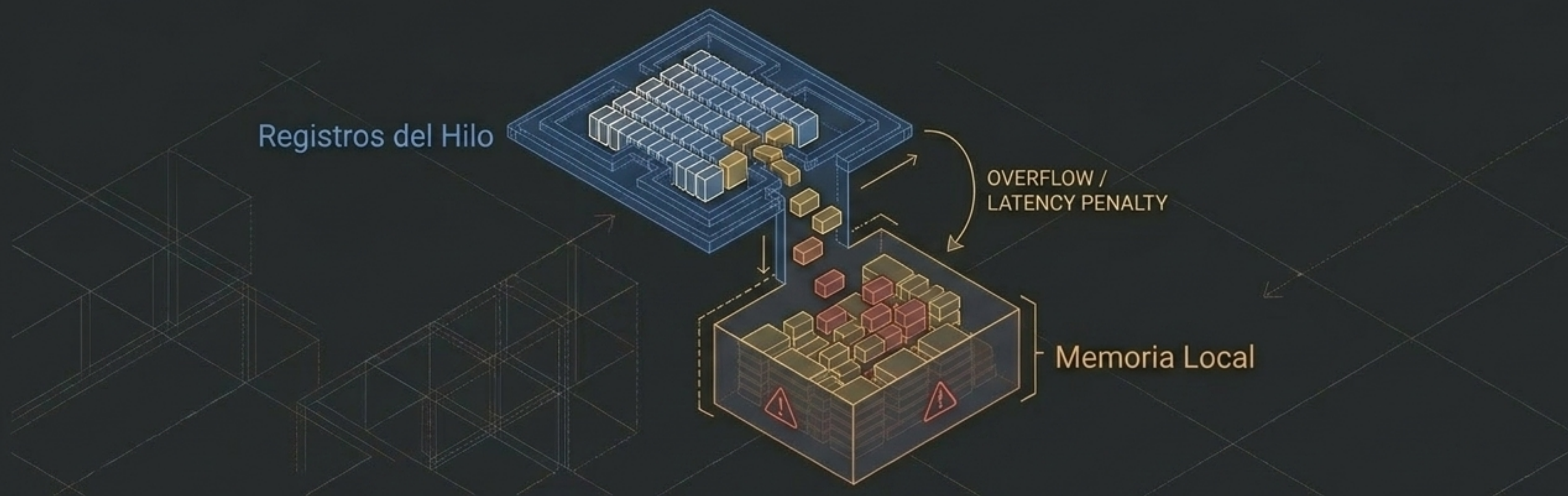
- Nivel más bajo y veloz de toda la jerarquía.
- Latencia cero en operaciones de lectura/escritura.
- Privados y exclusivos para cada hilo.
- Capacidad masiva a nivel global (Ej. 65,536 por SM), pero estrictamente racionados por cada hilo.

Memoria Local



- Estructura de almacenamiento tipo stack por hilo.
- Físicamente alojada fuera del núcleo (mitigado parcialmente hoy por cachés L1/L2).
- Considerablemente más lenta y penalizada.

El desbordamiento de variables destruye silenciosamente el rendimiento



El Fenómeno (Register Spilling)

Ocurre cuando un hilo requiere declarar más registros y variables locales de los que el Multiprocesador puede asignarle físicamente.

La Consecuencia

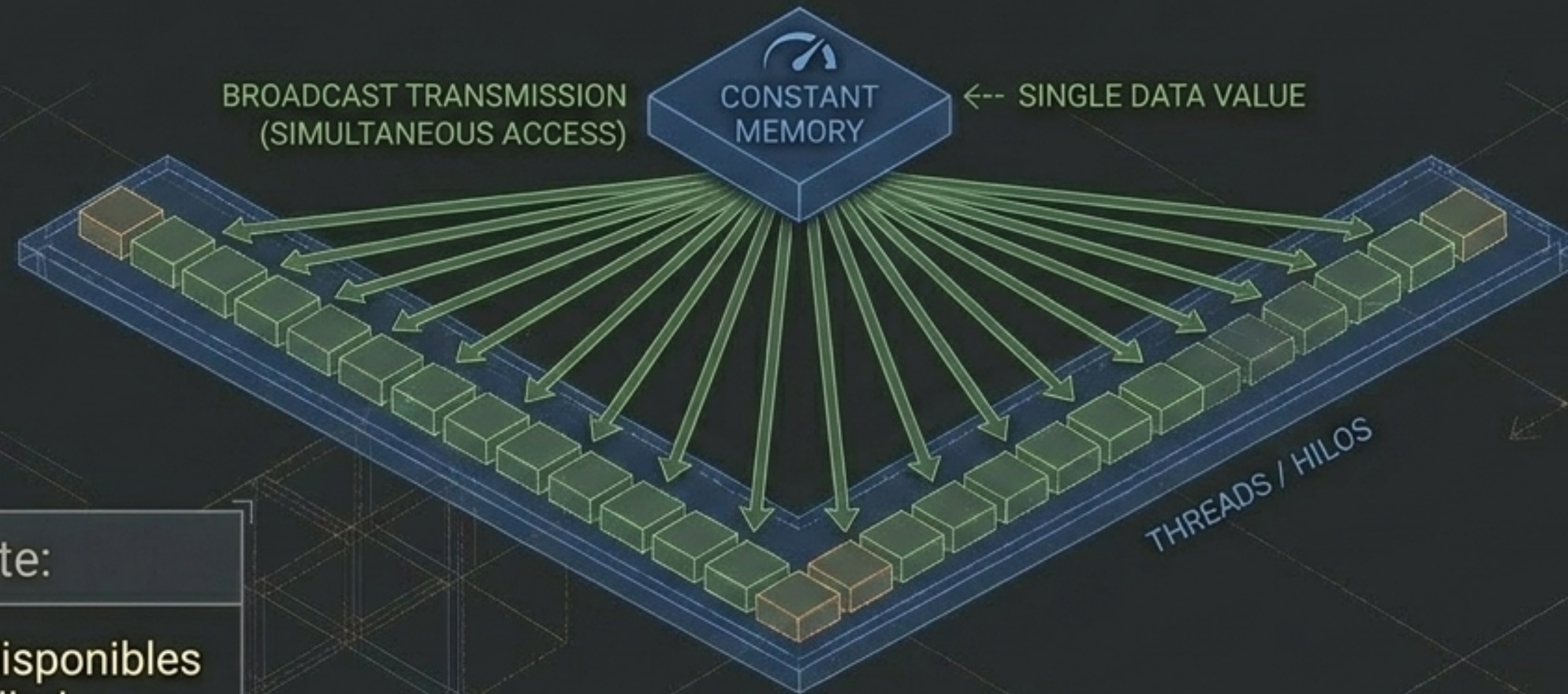
El compilador empuja forzosamente las variables sobrantes hacia la Memoria Local, multiplicando la latencia geométrica en cada cálculo.

La Auditoría Activa

Compilar usando `--ptxas-options -v,abi=no` para auditar kernels.

Forzar límites físicos con `--maxregcount N`.

La Memoria de Constantes transmite un único valor simultáneo a miles de hilos



Capacidad Límite:

- 64 KB totales disponibles para el desarrollador.

Comportamiento Especializado:

Altamente optimizada para Solo Lectura. Implementa hardware de Broadcasting: si un hilo solicita un dato, todos los hilos del bloque reciben el valor simultáneamente sin latencia adicional.

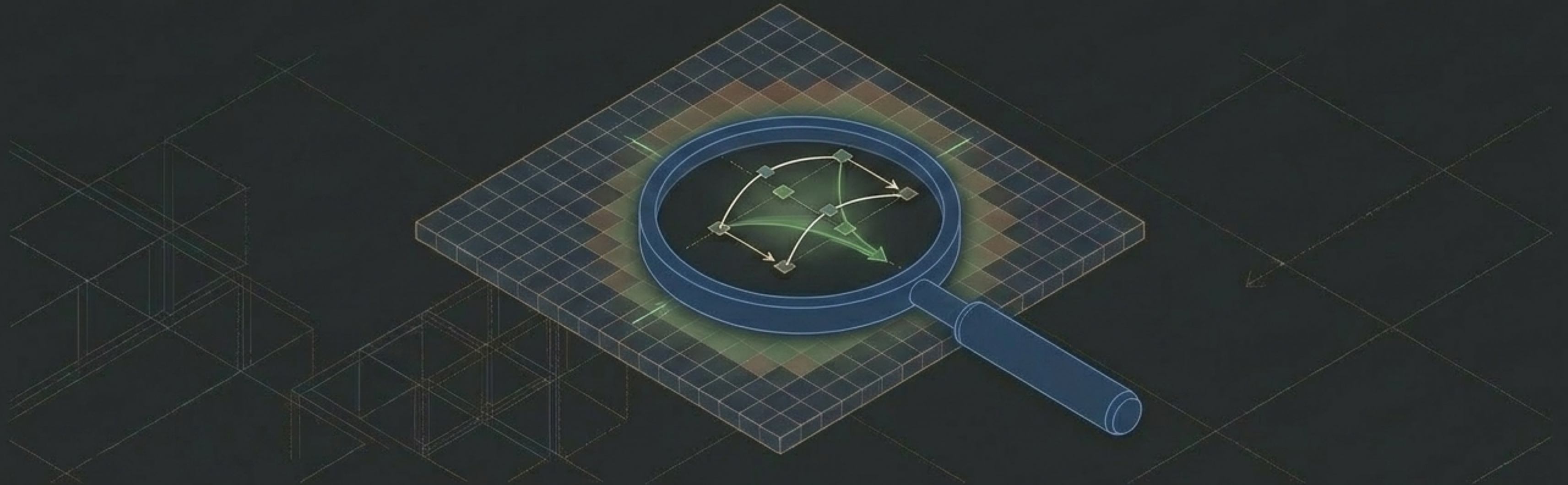
Implementación en Kernel:

```
__constant__ float coeficientes[10];
```

Transferencia desde el Host:

```
cudaMemcpyToSymbol(coeficientes, data, size);
```


Las Texturas rescatan el rendimiento espacial cuando la coalescencia es imposible



El Problema Geométrico

Las rotaciones o filtros de imagen exigen lecturas 2D dispersas que destruyen la linealidad y la localidad en la memoria global.

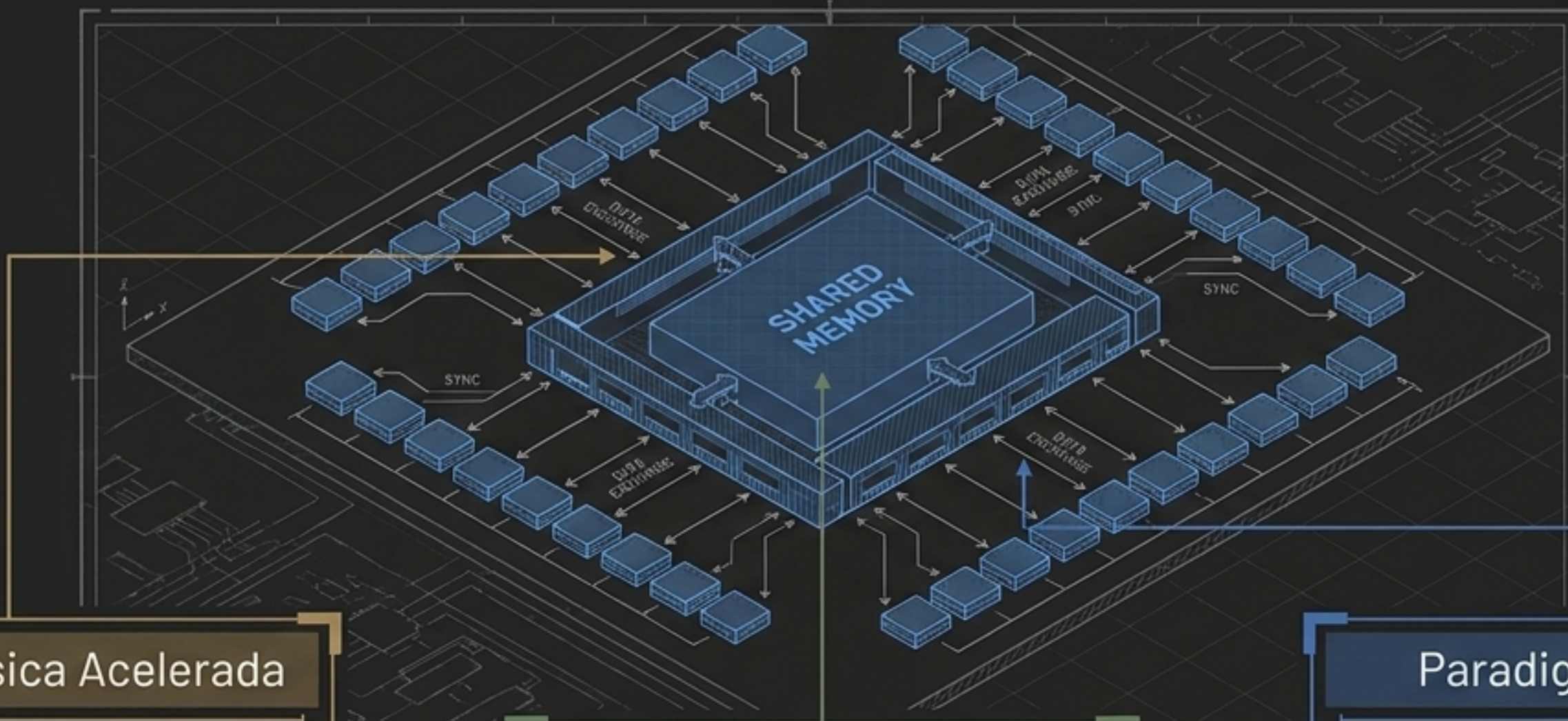
La Solución Arquitectónica

Uso de `cudaArray`. Memoria equipada con cachés dedicadas en hardware que entienden la proximidad espacial 2D y 3D nativamente.

Ventaja Computacional

Proporciona operaciones matemáticas integradas gratuitas, como interpolación bilineal y normalización, sin consumir ciclos de procesamiento en la ALU.

La Memoria Compartida es la mesa de trabajo colaborativa dentro del SM



Ubicación Física Acelerada

Reside físicamente dentro del chip SM. Es aproximadamente 10 veces más lenta que un registro, pero órdenes de magnitud más veloz que la DRAM Global externa.

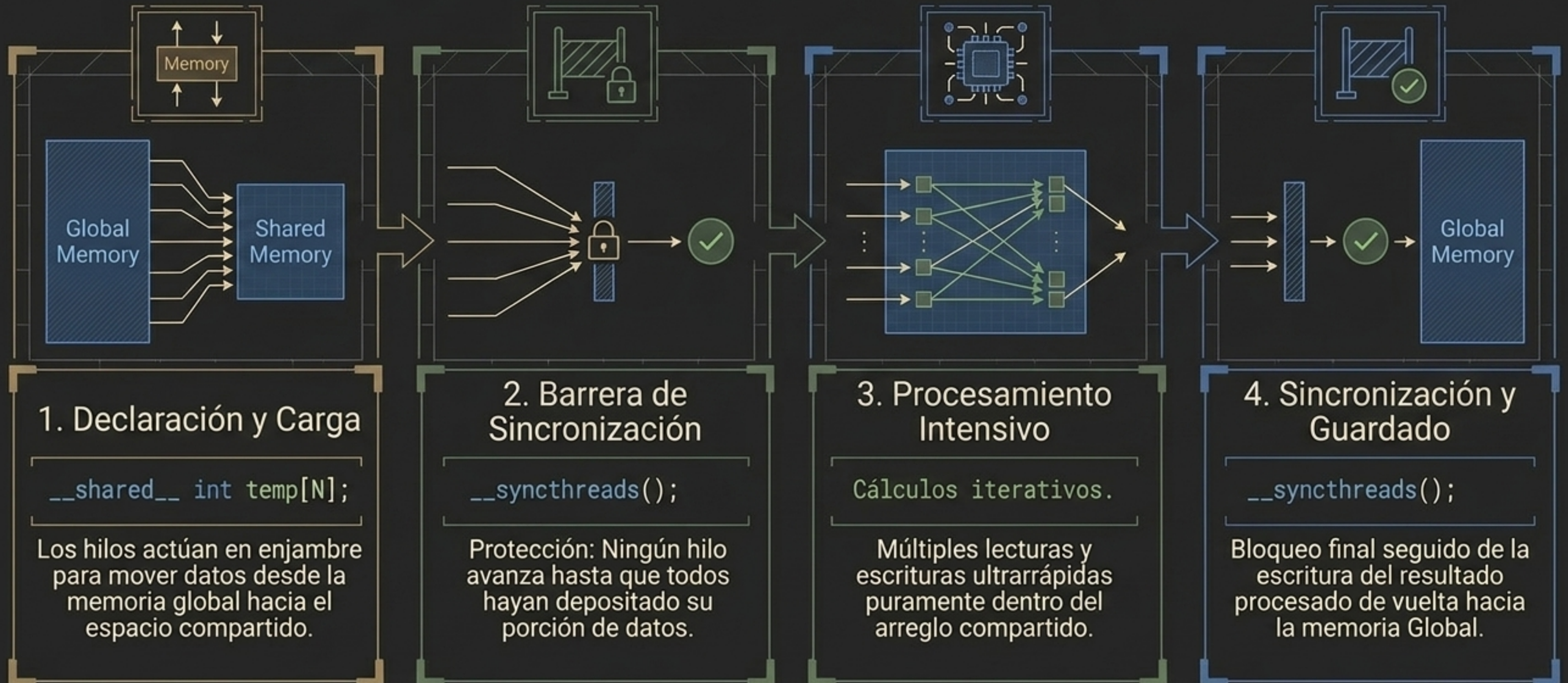
Propósito Exclusivo

Permite el intercambio directo de información y la comunicación sincronizada entre los hilos que pertenecen al mismo bloque.

Paradigma de Diseño

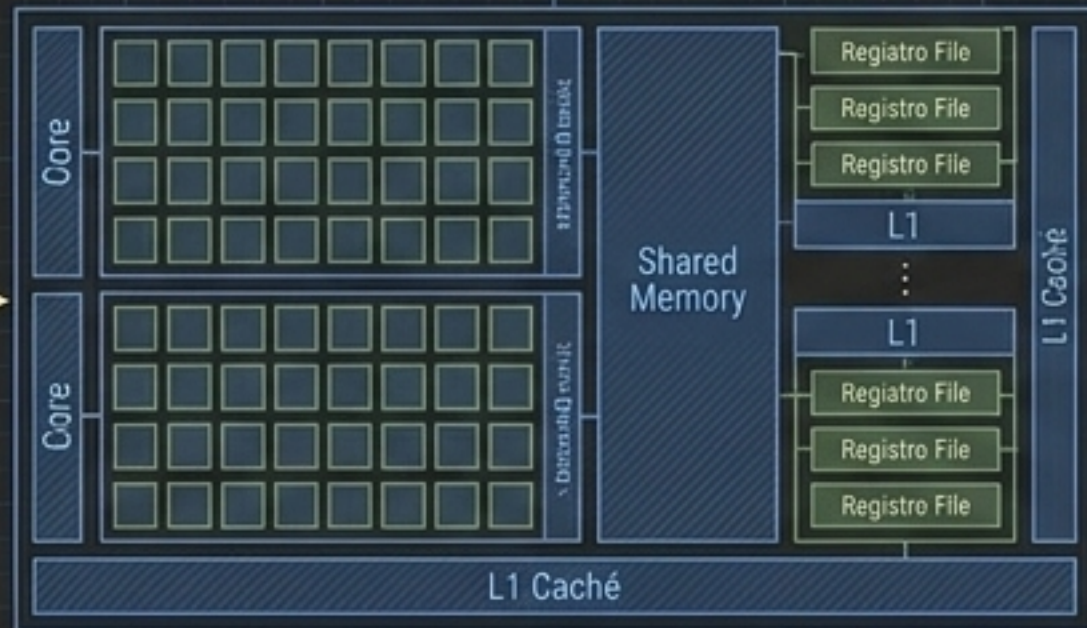
A diferencia de las cachés automáticas, es una caché de baja latencia gestionada estrictamente de forma manual por el desarrollador.

Orquestando la Memoria Compartida para evitar el caos concurrente



La anatomía del hardware impone un estricto equilibrio matemático

Especificaciones Físicas por SM (Ej. GTX 1080)



- Núcleos de procesamiento: 128 Cores
- Capacidad de Concurrencia: Máximo 2048 Hilos activos
- Almacenamiento Local Fijo: 64K Registros Totales | 96 KB Memoria Compartida | 48 KB Caché L1

La Ecuación de Rendimiento (Ocupación)

$$\frac{64,000 \text{ registros disponibles}}{2048 \text{ hilos requeridos}} = \sim 30 \text{ registros máximo por hilo.}$$

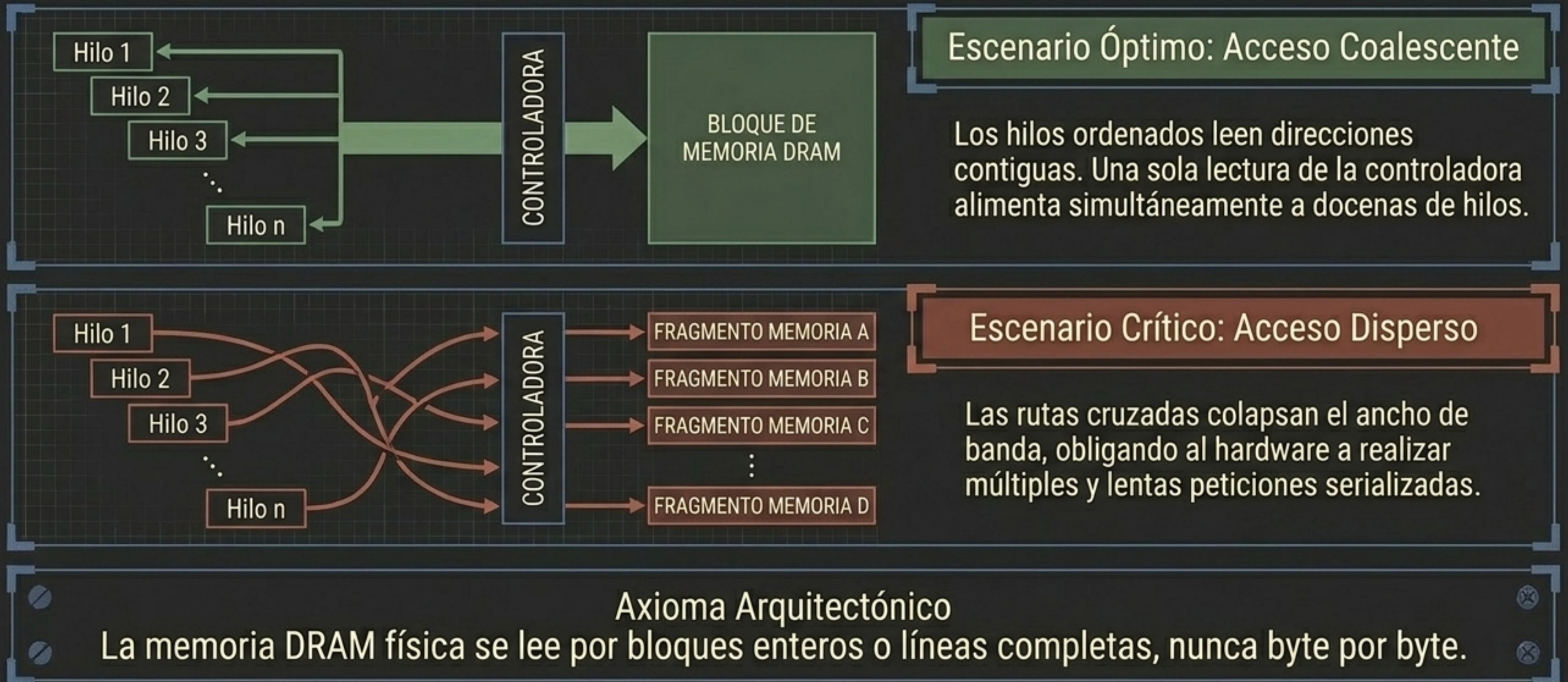
El Veredicto Arquitectónico

Si un kernel consume 60 registros por hilo en lugar del límite, el hardware se ve forzado a ejecutar solo la mitad de los hilos posibles, colapsando el paralelismo y el rendimiento general de la GPU.

Matriz de decisión táctica para la arquitectura de datos

Memoria	Latencia	Alcance	Ciclo de Vida	Permisos
Global	Altísima	Aplicación entera	Host manual	Lectura/Escritura
Compartida	Baja	Bloque de Hilos	Kernel	Lectura/Escritura
Constante	Baja (Broadcast)	Todos los hilos	Host manual	Solo Lectura
Textura	Caché 2D	Todos los hilos	Host manual	Interpolación
Registros	Cero	Hilo único	Ejecución de Hilo	Lectura/Escritura

El acceso coalescente expresa el ancho de banda masivo del bus DRAM



El paralelismo masivo exige un diseño consciente de la memoria

